Internship Report

2nd Year of Engineering Program

From May 19, 2025 to September 28, 2025

Last Name : Luydlin

First Name : Thibaud

Major : Electronics

Internship location : Politecnico di Bari, Bari, Italy

Internship supervisor : Prof. Guido MAIONE

Academic tutor : Prof. Valery LEBRET

Table of Contents

I- Presentation of the internship site 3

II- Context and objectives of the internship 4

III- Presentation of the planar robot 5

IV- Theoretical study of the robot 6

1-Modeling of the robot 6

2-Model on Matlab/Simulink 7

V- PID Controller 9

1- Linearization of the system 9

2- Determination of the parameters 10

3- Test of the controller 11

VI- Manual drawing of reference trajectories 12

VII- Pratical study on the robot 13

1- Microcontroller configuration 13

2- Implementation of a reference position 14

3- Robot manipulation 14

A- Application of torque 15

B- Application of the dynamic equation 15

C- Fallback method 16

VIII- Conclusion 17

Appendices 18

I– Presentation of the internship site

To complete my second-year internship and validate my 17-week international mobility, I carried out my internship at the Politecnico di Bari, in Italy.

The Politecnico di Bari is a public Italian university founded in 1990. It is the most recent Politecnico in Italy and also the only one located in the south of the country. It specializes in engineering, sciences, architecture, and industrial design. Despite being a relatively young institution, it has become a recognized reference institution [1].

The Department of Mechanics, Mathematics, and Management was recognized as a Department of Excellence at the national level by the Ministry of Education, Universities and Research for two consecutive periods : 2018–2022 and now 2022–2027. It ranks second nationwide in the field of Industrial and Information Engineering, just behind the Department of Management Engineering at the Polytechnic University of Milan.

Attached to the university are the Labzero laboratories, which bring together different research departments of the university: Electrical and Information Engineering, Mechanics, Mathematics and Management, as well as Civil Engineering and Architecture. A unit of the ENEA research center (the Italian National Agency for New Technologies, Energy and Sustainable Economic Development), based in Brindisi, is also integrated within it.

The Labzero laboratories are part of the regional research network of Apulia region and aim to integrating into other research and development networks. One of the main strengths of LabZERO is its ability to collaborate with actors from various sectors, such as industrial companies, local public organizations, and public administrations or municipalities. I completed my internship within the Labzero laboratories[2].

II- Context and objectives of the internship

This internship is part of the second year of the engineering program at ENSEIRB-MATMECA. Its purpose is to apply the knowledge and skills acquired during the first years of studies.

As part of our engineering program, we are required to complete a 17-week mobility period in a foreign country. I had the opportunity to carry out my intership in Italy, at the Polytechnic University of Bari, and more specifically in the Labzero laboratories located right next to the university.

For my second year, I chose Automation and Analog Electronics as my specialization, with the intention of continuing in the AM2AS spesialization (Automation and Mechatronics, Automotive, Aeronautics & Space). I wanted to find an internship in the field of automation. Thanks to my professor Pierre MELCHIOR, I was able to get in touch with Professor Guido MAIONE, who teaches in Bari. Professor MAIONE quickly agreed to host me for this internship. He is specialized, among other areas, in automation and control systems.

For the internship subject, Professor MAIONE assigned me the study of a two-degree-of-freedom planar robotic arm. The goal was to implement a control system enabling the robot to follow a predefined trajectory.

The objectives of my internship were first to carry out a theoretical study of the robot in order to establish its model, and then to validate this model through simulation. A second objective was to develop a way to design trajectories that would serve as reference inputs for the robot, in order to test trajectories of varying complexity. Finally, the work focused on implementing the model on a microcontroller to test it directly on the real robot and compare the theoretical results with the experimental results.

III- Presentation of the planar robot

The main objective of my internship was the control and automatic motion of a two-degree-of-freedom planar robotic arm, based on a manually drawn trajectory reference.

We can divide the robot into two parts : the mechanical part, which includes everything that moves and enables the motion of the robot, and the electronic part, which allows the control of the robot.

For the mechanical part, the robot consists of a fixed base and two arms. Arm 1 has a length l_1=17 cm, and Arm 2 has a length l_2=15 cm. Each arm is driven by a NEMA17 stepper motor with 200 steps per revolution. Each motor is equipped with a planetary gearbox: 10:1 for Arm 1 and 5:1 for Arm 2. Despite their simplicity, these motors were chosen for their compactness and efficiency, ensuring smooth motion and sufficient torque to move the arms. On the arms are also mounted magnetic incremental encoders, which measure the angular positions of the arms: θ_1, the angular position of Arm 1 with respect to the base, and θ_2, the angular position of Arm 2 with respect to Arm 1. In addition, each arm has a limit switch, which allows the robot to define a reference position (this point will be discussed later).

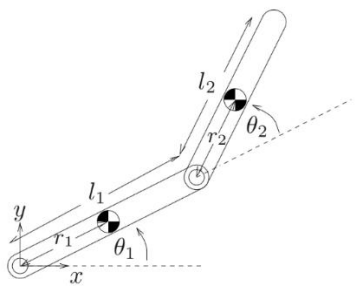

Figure 1: Kinematic diagram of the planar robot [3]

For the electronic part, controlling the robot, the system is equipped with a microcontroller: an STM32 Nucleo F411RE board, which constitutes the core of the electronic section. Two DRV8825 drivers are also used to control each motor by regulating the current, choosing the rotation direction of the motor, and setting the number of steps. These drivers are mounted on a shield, which also distributes power to the motors. The system is powered by an external 12V supply.

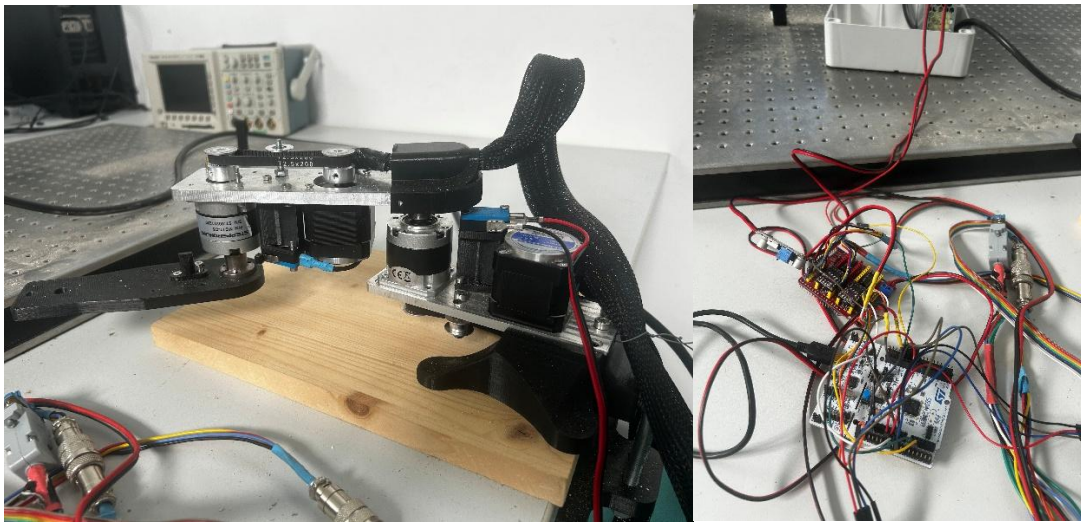

Figure 2: Picture of the robot and the components

Once the architecture of the robot was defined, I was able to set the different objectives mentioned previously, which would lead me to the final goal : to make the robot follow a manually defined trajectory.

All the codes used during my internship are provided in the appendices.

IV- Theoretical study of the robot

1- Modeling of the robot

To model the robot, we use the dynamic equation of robotic manipulators, which is the following: $\boxed{\tau=M(\theta)*\ddot{\theta}+C(\theta,\dot{\theta})*\dot{\theta}+G(\theta)}$. This equation makes it possible to obtain the torque to be applied to each motor to reach the desired position, depending on the angular position reference vector $\theta=[\theta_1;\theta_2]$, as well as its derivatives: the angular velocity $\dot{\theta}=[\dot{\theta}_1;\dot{\theta}_2]$ and the angular acceleration $\ddot{\theta}=[\ddot{\theta}_1;\ddot{\theta}_2]$. The equation is also composed of four matrix terms: the inertia matrix $M(\theta)$, the Coriolis matrix $C(\theta,\dot{\theta})$, which represents the forces due to rapid motions and rotation speeds of the robot, and finally the gravitational forces matrix $G(\theta)$ which we can neglect since the robot is in a horizontal position, meaning gravity has no effect [4].

The matrices take the following form:

$$M(\theta) = \begin{bmatrix} a + 2b\cos(\theta_2) & d + b\cos(\theta_2) \\ d + b\cos(\theta_2) & d \end{bmatrix} \quad C(\theta, \dot{\theta}) = \begin{bmatrix} -b\sin(\theta_2)\dot{\theta}_2 & -b\sin(\theta_2)(\dot{\theta}_1 + \dot{\theta}_2) \\ b\sin(\theta_2)\dot{\theta}_1 & 0 \end{bmatrix}$$

$$G(\theta) = \begin{bmatrix} (m_1 lc_1 + m_2 l_1)g\cos(\theta_1) + m_2 lc_2 g\cos(\theta_1 + \theta_2) \\ m_2 lc_2 g\cos(\theta_1 + \theta_2) \end{bmatrix}$$

, with :

- $a = I_1 + I_2 + m_1 l_{c1}^2 + m_2(l_1^2 + l_{c2}^2)$, $b = m_2 l_1 l_{c2}$, $d = I_2 + m_2 l_{c2}^2$
- m_1 and m_2 are the masses of arms 1 and 2
- $lc_1 = l_1/2$ and $lc_2 = l_2/2$
- I_1 and I_2 are the inertias of links 1 and 2 [4]

To determine these values, we measured the arms (both have roughly the same width and height: 5 cm width and about 7 mm thickness). Considering the materials (Arm 1 in metal and Arm 2 in 3D-printed plastic), along with the motor, encoder, and gearbox mounted on Arm 1, we mass the mass of Arm 1 to m_1= 1 kg and Arm 2 to m_2= 0.2 kg. With these masses, we can determine the inertias with the formula for a straight, homogeneous arm: $I = \frac{1}{3} * m * L^2$, we find I_1=0.00963kg.m^2 et I_2=0.0015kg.m^2. Once all these parameters are calculated, it is then possible to start the simulation phase [5].

<u>2- Simulation on Matlab/Simulink</u>

The first step in testing our model was to obtain a trajectory reference. To do this, I decided to draw the path that I wanted the robot to follow. I created a function that allowed me to draw this trajectory, which I will detail this part later. What matters here are the coordinates corresponding to each point of the trajectory. These coordinates make it possible to calculate the angular positions θ_1 and θ_2. For a two-joint planar robot, the coordinates are given by : $x = l_1\cos(\theta_1) + l_2\cos(\theta_1+\theta_2)$ and $y = l_1\sin(\theta_1) + l_2\sin(\theta_1+\theta_2)$. From these expressions, we can get : $\theta_2 = \arccos 2\left(\frac{x^2+y^2-l_1^2-l_2^2}{2l_1l_2}\right)$ and $\theta_1 = \text{arctan2}(y,x) - \text{arctan2}(l_2\sin\theta_2, l_1+l_2\cos\theta_2)$.[4]

In Matlab, I used the Robotics Toolbox, a library developed by the Australian roboticist Peter Corke. This library makes it possible to visualize the movement of the robot in Matlab and check whether the desired trajectory is physically possible.

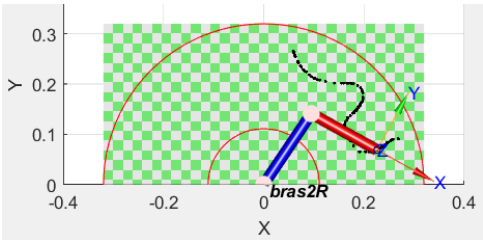

Figure 3: Example of a trajectory followed by the robot

From the formulas for θ_1 and θ_2, we can then obtain the reference curves for position, velocity, and acceleration.

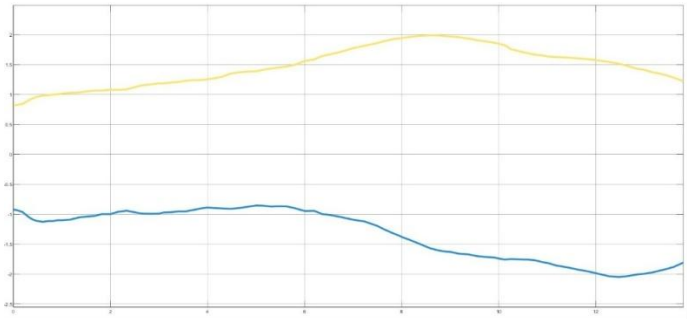

Figure 4: Position reference of arms 1 and 2 in radians

In Simulink, I used the dynamic equation of robotic manipulators in inverse dynamics mode, meaning that I used the known positions, velocities, and accelerations to calculate the torques. This was done by creating the corresponding block diagram (Figure 10 in the appendices), which allows to obtain the torques that must be applied to the robot in order to reach the desired position.

Once the torque was calculated in Simulink, the next step was to use t he dynamic equation of manipulator robots with direct dynamics. This means calculating the accelerations from the torques obtained previously, by isolating acceleration in the equation and then integrating it to obtain the position at the output. The resulting equation is $\boxed{\ddot{\theta} = M(\theta)^{-1}[\tau - C(\theta, \dot{\theta}) * \dot{\theta} - G(\theta)]}$. As in the inverse dynamics step, this equation was implemented as a block diagram in Simulink (Figure 11 in the appendices).

This Simulink diagram therefore allowed us to obtain the output position of the robot and to compare it with the reference in order to evaluate the error.

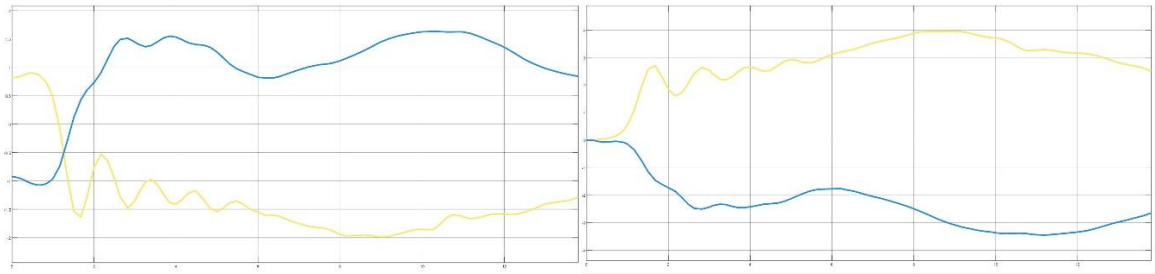

Figure 5: Output position and position error of arms 1 and 2

As we can observe, the reference is not followed at all in the output after applying direct dynamics to obtain the output position. The visible error varies between 2 and 4 radians, which is far too large to be ignored. This result was expected, since no correction system had yet been implemented to compensate for errors due to friction, inertia, or other effects. In order to correct this error, or at least reduce the gap between the reference and the output of the model, I chose to design a PID controller.

V– PID Controller

The error observed between the output position and the reference being too large to be ignored, which made it necessary to design a controller. For this purpose, I chose to use a PID controller.

1- Linearization of the system

To design the controller of the form $C(t) = K_p\, e(t) \;+\; K_i \int_0^t e(\xi)\; d\xi \;+\; K_d\, \frac{de(t)}{dt}$, or equivalently $C(p) = \left(K_p \;+\; \frac{K_i}{p} \;+\; K_d\, p\right) E(p)$ in the Laplace domain, where E(p) is the error between the reference and the output, it was first necessary to determine the parameters Kp, Ki, and Kd [6]. The first step was to linearize the dynamic equation of robot manipulator around a point. I chose to linearize it around the point (0,0). This choice sets the Coriolis matrix to zero C (0) = 0, and we can also neglect the gravitational matrix G since the robot operates in the (x,y) plane and gravity has no effect. We therefore obtain the following simplified equations for each arm :$\tau_1 = M_{11}\, \ddot{\theta}_1 \;+\; M_{12}\, \ddot{\theta}_2$ and $\tau_2 = M_{21}\, \ddot{\theta}_1 \;+\; M_{22}\, \ddot{\theta}_2$. To study each arm independently, we neglect $\ddot{\theta}_2$ in the first equation and $\ddot{\theta}_1$ in the second This leads us to the transfer functions : $\frac{\theta_1(p)}{\tau_1(p)} \;=\; \frac{1}{J_1 p^2}$ and $\frac{\theta_2(p)}{\tau_2(p)} \;=\; \frac{1}{J_2 p^2}$ with $J_1 = M_{11}$=0.0304, $J_2 = M_{22}$=0.026.

With these transfer functions, it becomes possible to determine the PID controller parameters.

2- Determination of the parameters

The determination of the controller parameters is based on an approach similar to that used during a previous project in the academic year. The idea was to set a desired phase margin M_φ in open-loop and a cut-off frequency w_u. I chose M_φ=60°, which provides a good compromise between speed and stability, and w_u =5rad/s. With a few lines of Matlab code, I was able to determine the controller and calculate the relevant parameters.

The controller was defined in the form: $K(p) = K_0 \cdot \frac{p/\omega_i+1}{p/\omega_i} \cdot \frac{p/\omega_1+1}{p/\omega_2+1}$ with

$\omega_i = \omega_u/5, \quad \omega_1 = \omega_u/\sqrt{a}, \ \omega_2 = \omega_{u\sqrt{a}} \ and \quad a = \frac{1+\sin\varphi_{max}}{1-\sin\varphi_{max}}$.

We recall that $\arg\big(K(j\omega_u)\, G(j\omega_u)\big) = -180° + M_\varphi$ and $|K(j\omega_u)\, G(j\omega_u)| = 1$, and we choose w_i=w_u/5, this means that the integral action of K introduces a phase of −11.3°. We can then calculate the controller.

The next step was to compute the phase lead a_1 from $a_1 = \frac{1+\sin(-180°+M_\varphi-pG_1+11.3°)}{1-sin(-180°+M_\varphi-pG_1+11.3°)}$ with pG_1 which is the phase of the transfer function, obtained using the Matlab bode() function, which also provides the gain. Then, the gain K_0 was calculated from :

$K_{01} = \frac{1}{\left(\frac{\sqrt{1+(\omega_u/\omega_{i1})^2}}{\omega_u/\omega_{i1}}\right)\cdot\left(\frac{\sqrt{1+a_1}}{\sqrt{1+1/a_1}}\right).modG1}$ with $\mathrm{mod}G_1 = \sqrt{(J_1\,\omega_u^2)^2}$ the module.

With these calculations, all the necessary elements were available to design the desired controller. We obtained the controller $K_1(p)$= $\frac{3.726\,p^2+6.794\,p+3.068}{0.8233\,p^2+25\,p}$. The open-loop Bode diagram of $K_1(p)*G_1(p)$ confirmed that the phase margin corresponds to the chosen value.

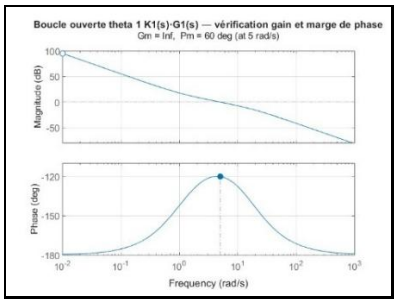

Figure 6: Bode diagram and phase of the open loop of θ_1

By applying the same method to arm 2, we obtained $K_2(p)$= $\frac{0.3187\,p^2+0.5811\,p+0.2624}{0.8233\,p^2+25\,p}$.

To simplify, we approximated $0.8233p^2 << 25p$, which gives:

$K_1(p)=\frac{3.726\,p^2+6.794\,p+3.068}{25\,p}$ and $K_2(p)=\frac{0.3187\,p^2+0.5811\,p+0.2624}{25\,p}$. We know that the general form of a controller is: $C(p) = \left(K_p + \frac{K_i}{p} + K_d\,p\right) E(p) = \frac{K_d\,p^2+K_p\,p+K_i}{p}\,E(p)$, we could identify the following parameters :

K_{p1} = 6.794/25 =0.272 ; K_{d1}= 3.726/25= 0.15 ; K_{i1} =3.068/25=0.123

K_{p2} = 0.5811/25 = 0.023 ; K_{d2}= 0.3187/25= 0.013 ; Ki_2 =0.2624/25=0.01.

We can now add the PID controller to our Simulink diagram to check whether it effectively corrects the error (Figure 12 in the appendices).

3- Test of the controller

As it is not recommended to use derivative blocks in Simulink, the derivative term was implemented by directly taking the difference between the reference and output velocities. The torque value obtained with the controller was then added to the torque calculated by inverse dynamics, which was the first part of the Simulink diagram (the complete diagram is shown in the appendices).

It is now possible to observe once again the error between the reference and the output in order to check whether the controller had the intended effect.

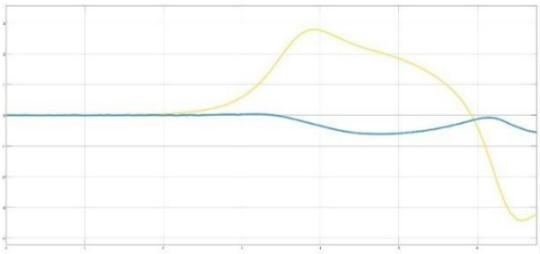

Figure 7: Error between the reference and the output position of arms 1 and 2

Although the error was reduced, it was still much too large, reaching almost 3 rad for Arm 1 and 0.5 rad for Arm 2. Even though 0.5 rad is smaller, it corresponds to about 30°, which is not negligible. The linearization around the point (0,0) explains the limited performance of the controller, since the robot rarely operates in this configuration. Any other linearization point would encounter the same issue, since the robot follows paths where the coordinates are constantly changing, making local linearization insufficient. To obtain acceptable performance, we tried different values for the coefficients until the error was sufficiently reduced. The chosen parameters were Kp_1 = 25, Kd_1 = 3.5, Ki_1 = 1.5, Kp_2 = 15, Kd_2 = 2.8, and Ki_2 = 1.5.

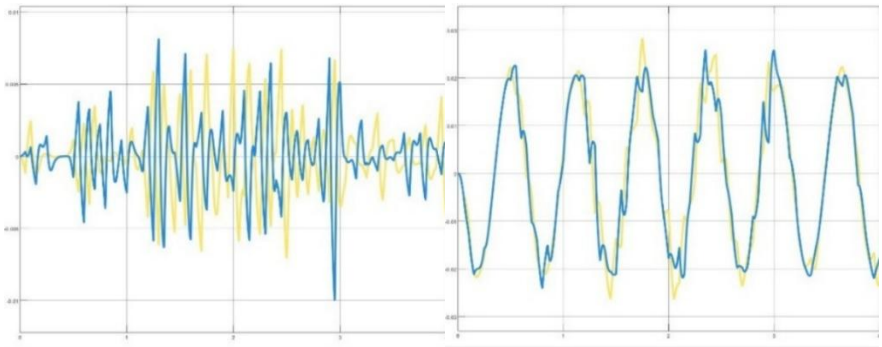

Figure 8: Error without disturbance and with disturbance

With these new values, the error between the reference and the output was reduced to almost zero, sometimes reaching only 0.01 rad on complex trajectories. Moreover, even when adding a disturbance (in this case, a 10 Hz sine wave of amplitude 1), the controller was able to cancel it almost completely, with the output amplitude limited to only 0.02 or 0.03 rad. These results validated both the model and the controller, making it possible to move on to the practical study on the real robot.

VI- Manual plotting of reference trajectories

Before moving on to the practical study on the robot, it was essential to define the reference trajectories that would be applied. To do this, I used Matlab to create a function that allows me to draw a custom trajectory. However, some limitations of the robot must be taken into account. We focused only on the upper part of the workspace, defining the base of Arm 1 as the point (0,0) in the (x,y) plane, and restricted the workspace to $y > 0$. In addition, I defined a semicircle of radius 32 cm, corresponding to the maximum reach of the robot. A small area around (0,0) was also excluded, in order to prevent Arm 2 from folding partially or completely onto Arm 1.

When a point was drawn beyond 32 cm or within the exclusion zone near (0,0), it was ignored and not stored in the array that saves the trajectory coordinates. The valid coordinates were then transformed into angular positions θ, which served as the references for simulation or for the real robot.

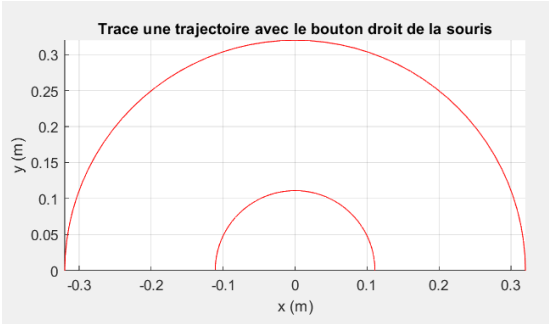

Figure 9: Workspace area for trajectory plotting

Moreover, to ensure that there would be no collision between the arms or the components such as the motors, gearboxes, or encoders mounted on the arms and the base, the arms were not allowed to enter the lower half-plane. For example, a point at (0.2,0) could result in part of each arm extending into the lower zone if Arm 2 was positioned to the left of Arm 1. To avoid this, we impose a condition : whenever the point of the trajectory was located on the right side of the plane, Arm 2 had to remain on the left of Arm 1, and when the point was on the left side, Arm 2 had to remain on the right. The transition occurred when successive points x_n and x_{n+1} had opposite signs, which meant crossing x = 0. At that instant, the position of Arm 2 changes.

In addition, to avoid generating inconsistent torques (either too low or too high), it is necessary to take into account the mechanical limits of the motors. These can withstand a maximum torque of 0.45 N·m, which corresponds to 4.5 N·m for arm 1 and 2.25 N·m for arm 2 after the reduction ratios of the planetary gearboxes (10 :1 and 5 :1, respectively). I therefore impose a time step of 25 ms between each point of the trajectory: this choice ensures that the robot has enough time to reach the next point, even when they are far apart, and prevents the generation of excessive torques caused by overly abrupt accelerations. (Drawing zone : see Figure 18 in the appendices) [7].

Once the trajectory was drawn, the coordinates were saved, converted into angular positions, and stored in an array. These values were then used as reference inputs for the simulation and the real robot.

VII- Practical study on the real robot

1- Microcontroller configuration

Before manipulating the robot, it was important to discuss the software aspect. To control the robot, I used an STM32 microcontroller programmed in C with the

STM32CubeIDE software. The first step in this software was to configure the different pins of the board so that each pin would be assigned a specific function. Some of these pins are used to generate PWM signals, which control the speed and direction of motor rotation. Others are configured as GPIO inputs to read the states of the limit switches, which define a reference position. Timers are also used to measure angular positions via the incremental encoders. Finally, a UART communication link is configured to ensure data exchange between the microcontroller and the computer. This configuration of the pins on the board was a necessary foundation for the proper functioning of the robot.

2- Implementation of a reference position

At each restart, the encoder values are reset to zero. This means that if the robot finishes a trajectory with Arm 1 at $\theta_1 = 1.2$ rad, this position would be considered as 0 rad at the beginning of the next operation. This would cause a mismatch between the actual position and the measured position. To avoid this issue, it was essential to define a reference position that the robot would reach at the beginning of each new trajectory. The chosen reference position corresponded to $\theta_1 = 0$ and $\theta_2 = 0$, meaning that the robot would be fully extended to the right of the base, equivalent to the point (0.32, 0) in the (x,y) plane.

To achieve this, I applied torque to the motors until the arms reached the limit switches. Once the switches are reached, the motors stop. Then, I applied a precise torque for a defined duration to bring the robot into the reference position. At that moment, the encoders reset this position as the origin. From then on, we focus only on the manipulation of Arm 1.

3- Robot manipulation

Before starting robot manipulation, it is important to clarify that the microcontroller does not directly apply torque to the motor. The theoretical torques are first converted into angular velocities, then into pulse frequencies, and finally into ARR (Auto-Reload Register) values. The ARR defines the value up to which the timer counter increments before resetting to zero : the smaller the ARR, the higher the pulse frequency, and the faster the motor rotates, and vice versa. (The detailed formulas are presented in the appendices).

Each new reference trajectory does not necessarily start from the reference position. Therefore, the arm had to first move to the initial point of the trajectory before starting to follow it. To do this, we apply a light torque to Motor 1, while constantly measuring the

position and comparing it with the first reference point. When the difference between the two is less than 0.01 rad, the motor stops, and trajectory tracking can begin.

Two different methods were implemented to make the robot follow the reference.

A- Application of torque

In the first method, the microcontroller directly applies the torques calculated previously with Matlab/Simulink. These values are exported into a .h file integrated in the STM32CubeIDE project, which made them directly usable in the C code of the microcontroller. This method has the advantage of producing a very lightweight and simple C code. However, it also has a major drawback : it is not possible to apply correction during execution, since torques rather than positions are used as the reference. Any errors could therefore not be compensated. Since the torques were calculated theoretically, they might not perfectly match the behaviour of the real robot, which was the main limitation of this method.

Another problem appeared during implementation : the motor cannot handle overly rapid variations of ARR, which is directly linked to rotation speed. Since the calculated torques can vary significantly between two consecutive points of the trajectory, the ARR value can jump from a few thousand to several hundred thousand in a very short time. These sudden variations cause the motor to stop abruptly, preventing it from following the imposed trajector. This constitutes the main experimental limitation of this first method.

B- Application of the dynamic equation

The second method is based on an embedded systems logic, where Matlab is only used to generate the reference trajectory. The inverse dynamics calculations and error correction were carried out directly on the microcontroller. The STM32 received, via a .h file, the reference values of position, velocity, and acceleration, then used the robot's dynamic equation to compute the necessary torques. Moreover, the microcontroller applied a PID controller, using the error between the position measured by the encoder and the reference. This method required more resources and a more complex code, but it allowed the robot to operate in a truly autonomous way. It fit perfectly within the framework of embedded systems and real-time control.

Unfortunately, this method suffers from the same issue as the first one. The inverse dynamics allow for torque calculation, but these torques are again converted into ARR values.

The problem of rapid ARR variations therefore remains, once again causing the motors to stall. Thus, despite its theoretical relevance and efficiency potential, this method could not be validated under real conditions.

C - Fallback method

Since the use of the two previous methods did not succeed, it was necessary to find an alternative method to make the robot move as desired. For this, we used a method inspired by the one applied to bring the robot to the first reference position. This method is simple : at each point of the trajectory, the encoders measure the position, and the microcontroller compares it with the reference input. A constant ARR is then applied in the correct direction as long as the error is greater than 0.01 rad. This approach offers several advantages: it eliminates ARR variations and prevents motor stalls. Moreover, it guarantees that the reference positions will be reached, since the motor only stops when a position is achieved, then restarts to move toward the next point. This also makes it possible to record the positions to plot the error between the reference and the output.

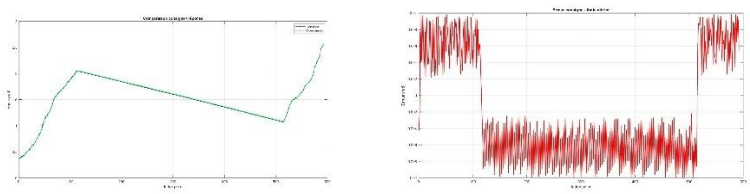

Figure 10 : Courbe de consigne/sortie et d'erreur

The error curve clearly shows that the trajectory is respected with a small error, which validates this method for simple manipulations. However, this solution has significant limitations. First, the movements of the robot are discontinuous and jerky: the motor moves toward the position, then stops, then restarts, resulting in a loss of fluidity and a behavior that is not representative of real-time trajectory tracking. Next, this method completely ignores physical constraints such as inertia, gravity, and friction, which are not compensated. This leads to reduced accuracy in the presence of external disturbances. Since the two arms are controlled separately with similar ARR values, they almost never reach their target positions at the same time. For example, one position may be reached in 50 ms while another may take 200 ms. A trajectory point (x,y)(x,y)(x,y) that requires both angles to be reached simultaneously is therefore almost never achieved.

This method is thus an effective fallback solution to validate the robot's basic operation, but it remains unsuitable for precise and continuous tracking. It may be appropriate

for very simple systems with a single arm, for instance, but it does not represent a sustainable solution for a multi-arm robot expected to follow complex trajectories.

Conclusion

On a personal level, completing my internship at the LabZERO laboratories of the Politecnico di Bari allowed me to immerse myself in a professional and international work environment. This experience gave me insight into the operation of a specialized research laboratory and granted me significant autonomy in carrying out a technical project. Doing this internship abroad also enabled me to develop adaptability and communication skills in an international context.

On a technical level, the internship allowed me to apply all the steps involved in the operation of an embedded system : modeling, simulation, and then testing on the real robot. The first part focused on the dynamic study of the robot and its modeling in Matlab/Simulink, the second on the design of a PID controller to validate the model in simulation, and finally the last part on real experimentation with different methods : direct application of calculated torques, then on-board torque computation via inverse dynamics and PID correction.

Although promising in theory, these methods did not work in practice due to the rapid variations of the ARR, which caused the motors to stall. To address this issue, an alternative method was implemented. While effective for a simple system, it presented major limitations, such as jerky movements, lack of synchronization between the two arms (preventing trajectory points from being reached), and no consideration of physical disturbances.

Beyond these disappointing results on the robot, this internship taught me the difference between a theoretical model and a real experiment, and it helped me develop the rigor essential for working on complex systems. Finally, it confirmed my interest in automation and control of systems, while also enriching my human and professional experience.

APPENDICES

Addendum "Économy and Management"

For the internal and external environmental analysis of the internship site, I chose the theme of sustainability. The LabZERO laboratories, attached to the Politecnico di Bari, are multidisciplinary research facilities dedicated to the development of new technologies in the fields of renewable energy and energy efficiency, both in industrial and urban contexts. LabZERO's research activities are organized into four units: DEI (Department of Electrical and Information Engineering), DMMM (Department of Mechanics, Mathematics and Management), DICAR (Department of Civil Engineering and Architecture), and the ENEA research center (National Agency for New Technologies, Energy and Sustainable Economic Development) based in Brindisi. These four units provide a very broad research scope and make it possible to apply the laboratories' environmental vision to a wide range of domains.

During the research and experimentation phases, the laboratories use high-quality equipment in order to minimize risks related to applied research, such as the risk of damaging a prototype or expensive equipment. They also support the development of a product from its design stage to potential commercialization.

The laboratories also conduct research in the development of complex systems in the fields of energy efficiency, energy conservation, sustainable manufacturing, and smart cities. All of this is done with a forward-looking approach and an ecological perspective at every scale.

An important element to consider lies in the research methods used. In particular, the laboratories employ advanced simulation techniques such as Hardware-in-the-Loop (HIL) and Power-Hardware-in-the-Loop (PHIL). These real-time simulation approaches involve connecting a physical component to a virtual environment that simulates an entire complex system. This enables a large part of the experiments that would normally need to be performed on the real system to be carried out virtually. Such methods save considerable time, since experiments can be conducted continuously, 24 hours a day if desired. In addition, the use of these simulations significantly reduces costs, in particular by eliminating the need to build physical prototypes and the associated software. This also offers environmental benefits by limiting the production of additional hardware and the consumption of raw materials. Moreover, these methods increase the success rate of experiments, making it possible to identify the most efficient solutions more quickly, and therefore reducing the number of real-world trials required to achieve equivalent results. This simultaneously reduces financial costs and the environmental footprint [8].

On a larger scale, LabZERO works closely with around forty partners, including industrial developers, local authorities, and municipalities. This ensures that the progress achieved in the laboratories has a nationwide impact on sustainable development. Several projects developed at LabZERO aim to extend beyond the academic sphere, such as micro-wind turbines, solar heating and cooling systems, the development of smart materials and devices for thermoelectric production from residual heat or solar energy, as well as infrastructure for electric vehicles, including smart charging and Vehicle-to-Grid systems. This last technology enables rechargeable electric vehicles not only to draw electricity from the grid like standard EVs, but also to inject electricity back into the grid when needed. This contributes to balancing the power grid: vehicles can reinject electricity during peak consumption and prioritize charging during off-peak periods. Thanks to this functionality, the grid becomes more stable and less prone to disturbances.

All these projects on which LabZERO is working aim both to achieve cost savings and to generate a significant positive impact on the environment, on a scale far beyond that of the laboratories themselves [2][9].

This environmental analysis of the LabZERO laboratories clearly shows their commitment into sustainable development. Their research focuses on energy efficiency, renewable energy, and recycling. The methods they employ, such as HIL simulations, are also designed to minimize ecological impact. Finally, the laboratories' collaborations with companies and local authorities ensure that their innovations contribute to ecological transition on a national scale.

This illustrates how university laboratories can simultaneously carry out scientific research and improve the environmental impact of their work.

Bibliography

[1] Contributori di Wikipedia. (2025, 5 juin). *Politecnico di Bari*. Wikipedia. https://it.wikipedia.org/wiki/Politecnico_di_Bari#:~:text=Il%20Politecnico%20di%20Bari%20%C3%A8,ingegneria%20e%20del%20disegno%20industriale.

[2] *LabZERO | Politecnico di Bari*. (s. d.). https://research.poliba.it/labs-networks/labzero

[3] *Feedback control of two-link planar manipulator*. (2015, 1 novembre). Engineering Stack Exchange. https://engineering.stackexchange.com/questions/6006/feedback-control-of-two-link-planar-manipulator

[4] Foiegreis. (2024, 20 mai). Robotics Part 26 - Euler-Lagrangian Formulation of Dynamics. *RoboticsUnveiled*. https://www.roboticsunveiled.com/robotics-dynamics-lagrangian-formulation/#Euler-Lagrangian_Formulation_of_Dynamics

[5] ViLAP. (2022, 18 novembre). *moment d'inertie 3 : Etablissement du moment d'inertie d'une tige rectiligne et homogène* [Vidéo]. YouTube. https://www.youtube.com/watch?v=b7LOBOSiJ6Q

[6] *Régulateur PID — Wikipédia*. (2024, 16 décembre). https://fr.wikipedia.org/wiki/R%C3%A9gulateur_PID

[7] J.Touassilt. (s. d.). *17HS15 1504 ME1K Full datasheet*. Scribd. http://fr.scribd.com/document/872528448/17HS15-1504-ME1K-Full-Datasheet

[8] *What is Hardware-in-the-Loop Testing?* (2025, June 3). Ansys. https://www.ansys.com/simulation-topics/what-is-hardware-in-the-loop-testing

[9] Contributeurs aux projets Wikimedia. (2025, 15 juin). *Véhicule-réseau*. https://fr.wikipedia.org/wiki/V%C3%A9hicule-r%C3%A9seau

Simulink diagrams

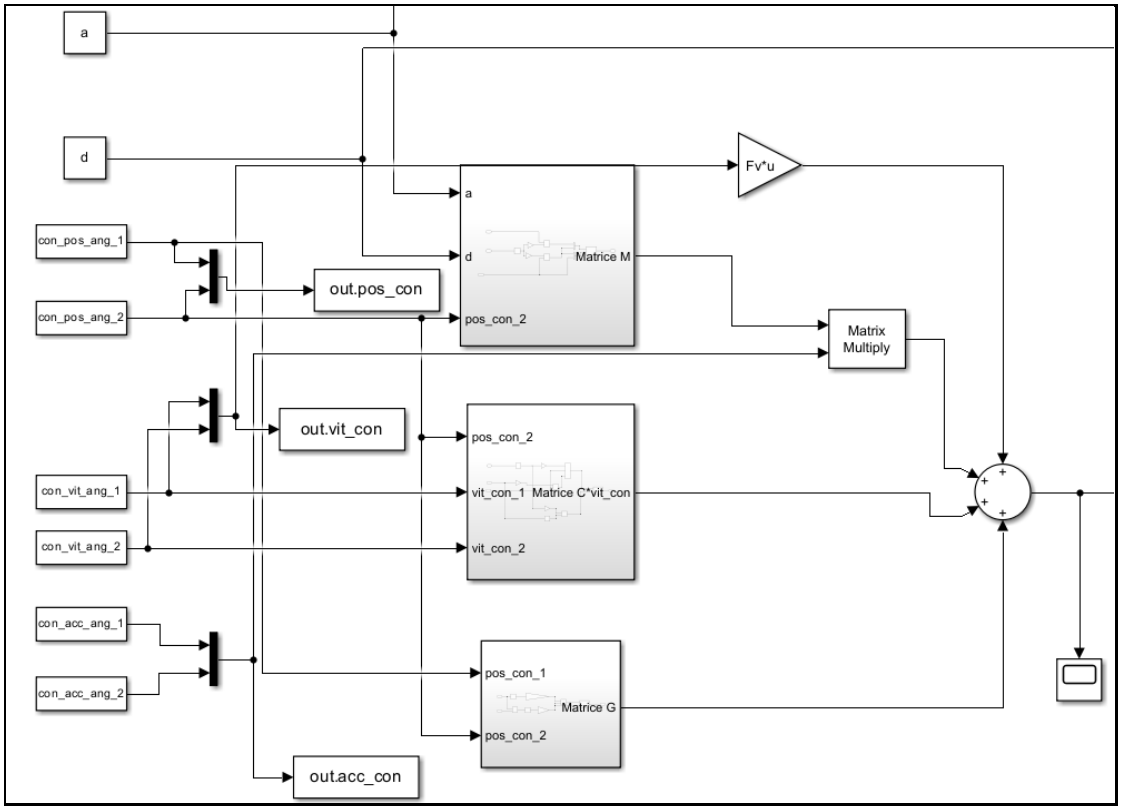

Figure 11 : Simulink diagram of inverse dynamics

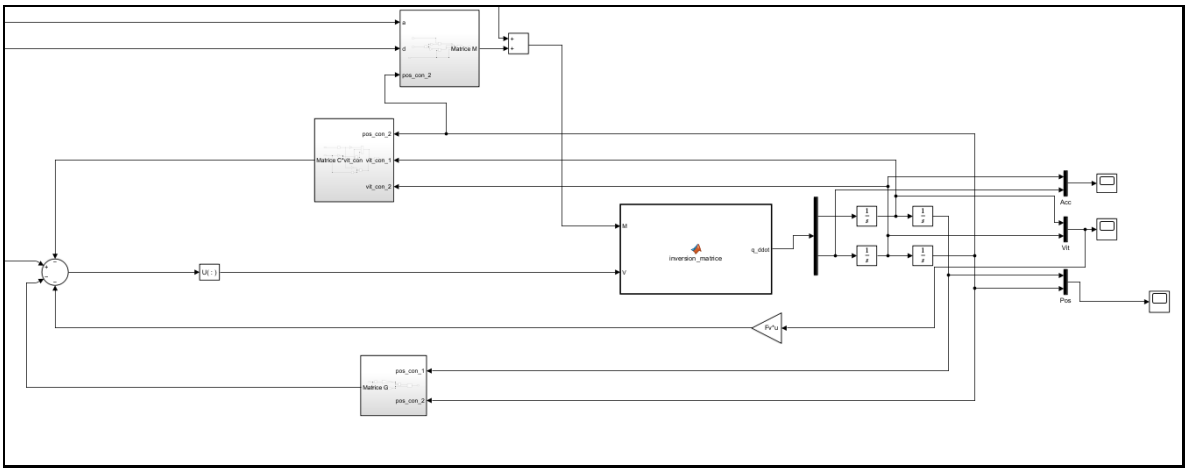

Figure 12 : Simulink diagram of direct dynamics

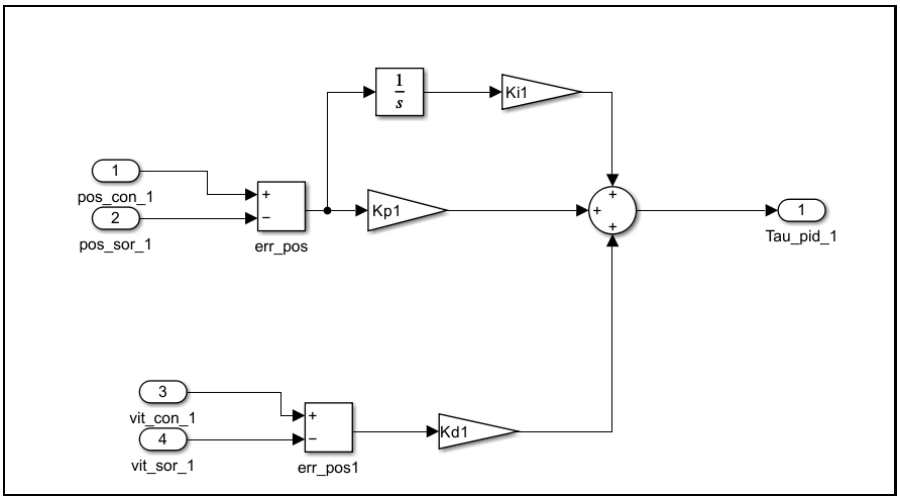

Figure 13 : Simulink diagram of the PID controller

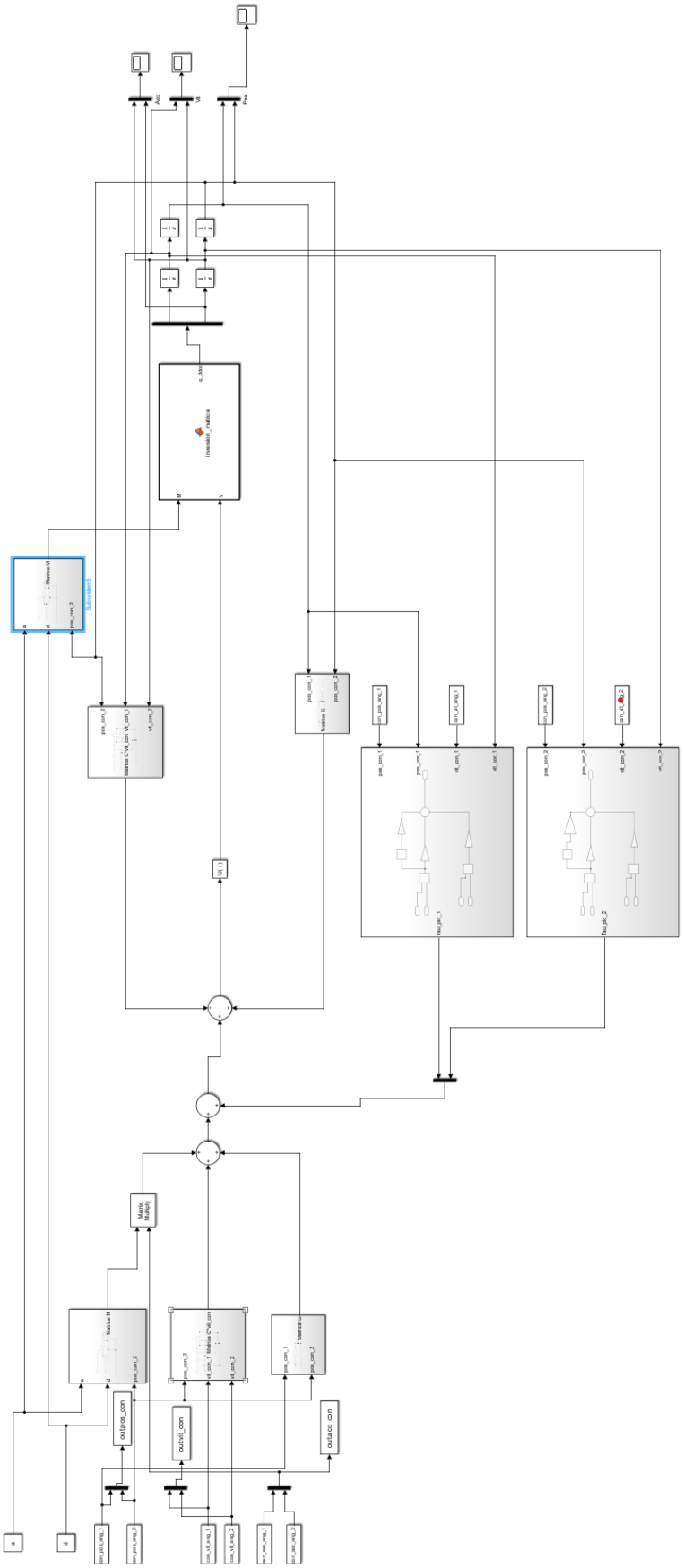

Figure 13 : Complete Simulink diagram

Various figures :

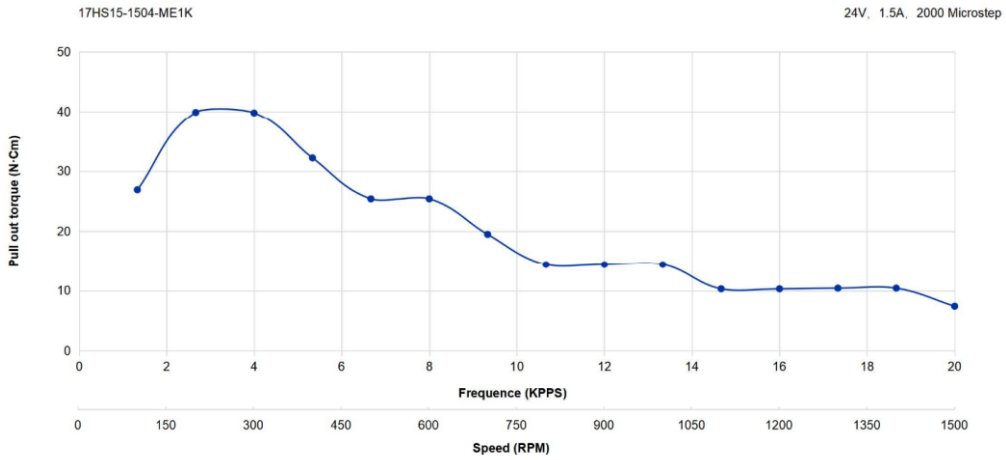

Figure 14 : NEMA17 motor characteristic curve

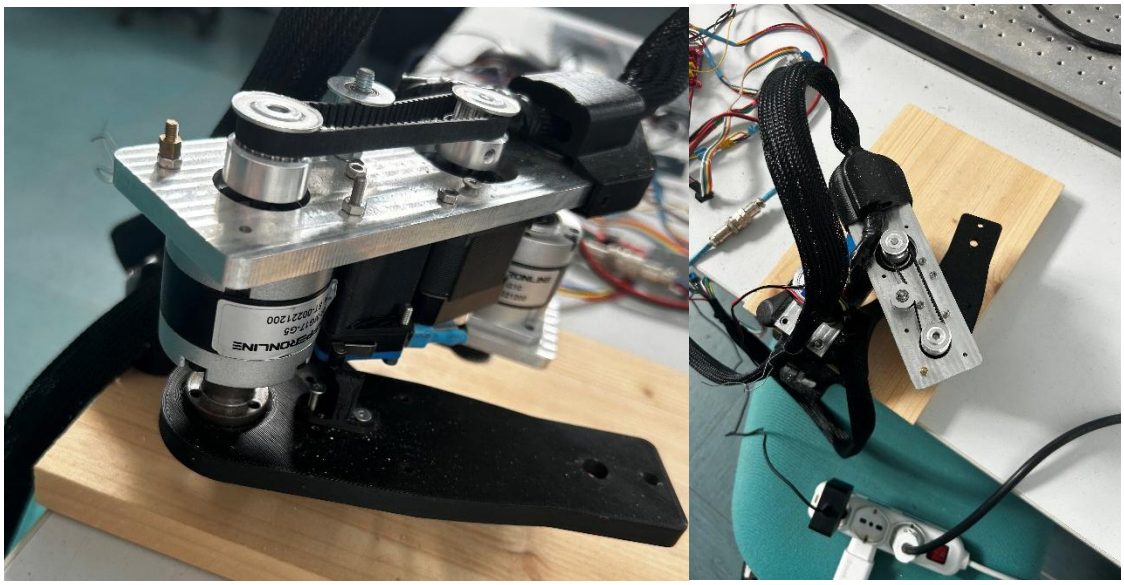

Figure 15 : Robot in end position

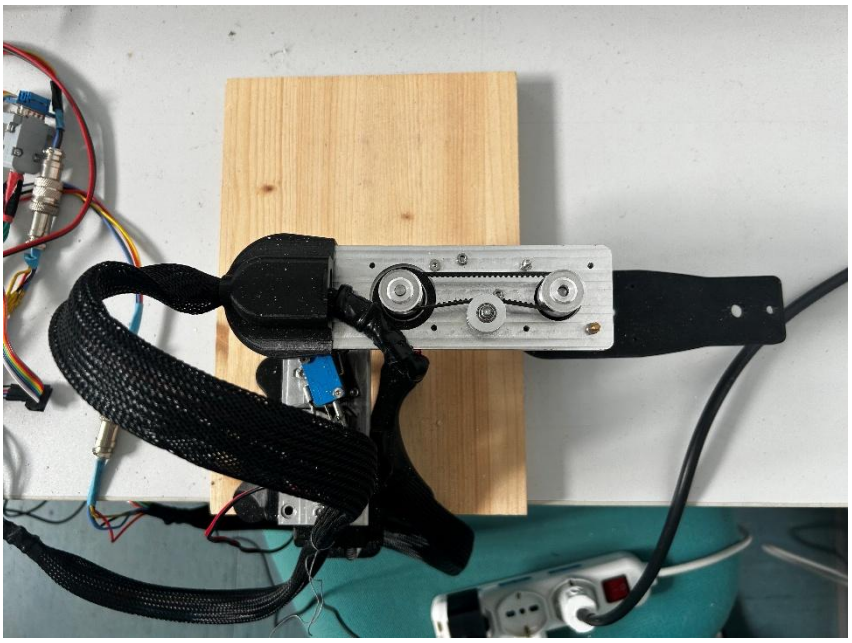

Figure 16 : Robot in reference position, unfolded to the right

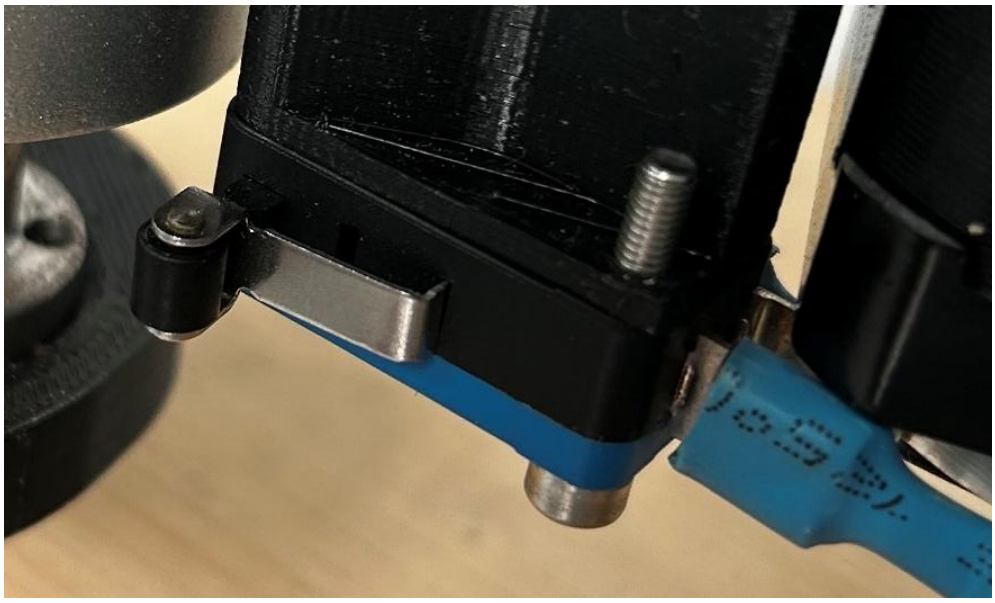

Figure 17 : Stop for arm 1

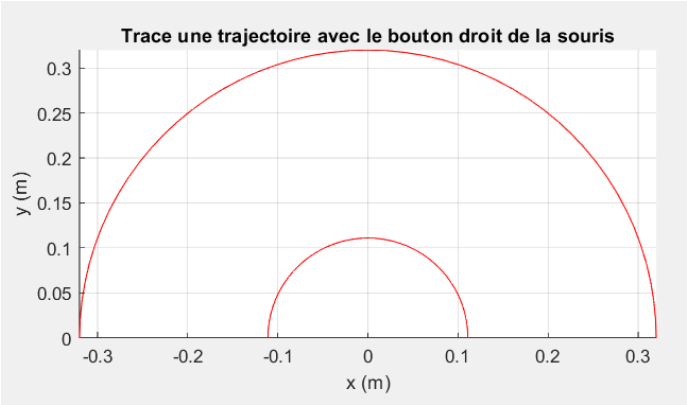

Figure 18 : Manual trajectory drawing area

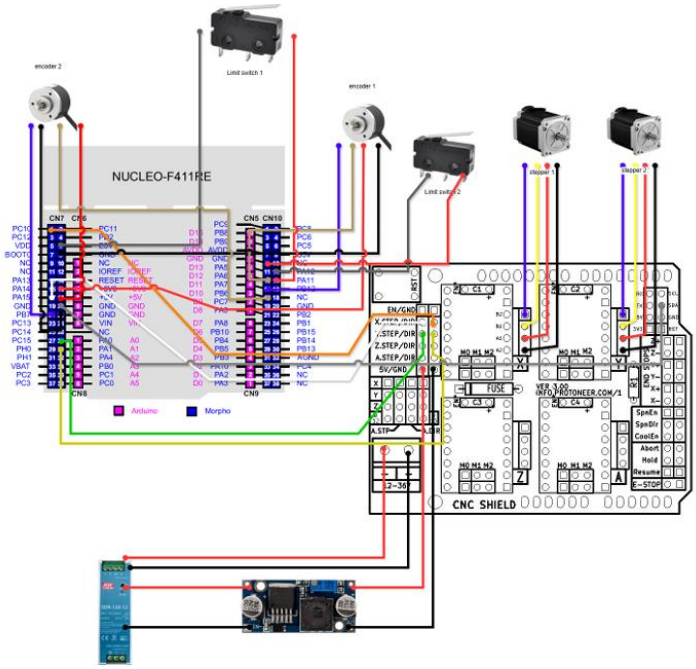

Figure 19 : *Wiring diagram of the robot taken from the report by the students who built it*

<u>Formulas Torque → ARR :</u>

$$\omega = \frac{\tau}{\tau_{max}} \cdot \omega_{max} \ , \ f = \frac{\omega \cdot N_{steps}}{2\pi} \ \text{ et } ARR = \frac{84 \cdot 10^6}{2 \cdot f}$$

With tau is the torque, w i s the angular velocity, N_{Steps} is the number of steps for a complete rotation of an arm and the constant 84.10^6 corresponds to the clock frequency (84 MHz) of the STM32F411 microcontroller, used as the time base for the timers.

Matlab code

Main code :

```
clc; clear; close all;
clear donnees_trajectoires;

pas_temps = 0.025;

% Paramètres du robot
l1 = 0.17; l2 = 0.15;
lc1 = l1/2; lc2 = l2/2;
m1 = 1; m2 = 0.2;
I1 = 0.00963; I2 = 0.0015;
g = 9.81;
fv1 = 0.04;
fv2 = 0.03;

% Coefficients des matrice
a = I1 + I2 + m2*(l1^2 + lc2^2) + m1*lc1^2;
b = m2 * l1 * lc2;
d = I2 + m2*lc2^2;
%Coefficients des correcteurs PID
Kp1 = 25; Ki1 = 3.5; Kd1 = 1.5;
Kp2 = 20; Ki2 = 2.8; Kd2 = 1.5;

afficherRobot = false; % Afficher ou non l'animation du robot

% Interpolation pour un pas constant
[xDraw, yDraw] = dessinTrajectoire(); %Permet de tracer la trajectoire
nb_points = length(xDraw);

t = (0:nb_points-1) * pas_temps;

%Lisser la trajectoire entre les points
xPath = interp1(linspace(0,1,length(xDraw)), xDraw, linspace(0,1,nb_points), 'linear');
yPath = interp1(linspace(0,1,length(yDraw)), yDraw, linspace(0,1,nb_points), 'linear');

% Cinématique inverse
pos_ang1 = zeros(size(t));
pos_ang2 = zeros(size(t));
for i = 1:nb_points
    x = xPath(i);
    y = yPath(i);
    r = sqrt(x^2 + y^2);

    if r > l1 + l2 || r < abs(l1 - l2)
        if i > 1
            pos_ang1(i) = pos_ang1(i-1);
            pos_ang2(i) = pos_ang2(i-1);
        end
        continue;
    end
```

```
    D = (x^2 + y^2 - l1^2 - l2^2) / (2*l1*l2);
    D = min(max(D, -1), 1);
    %Permet de définir la position du bras 2 par rapport au bras 1
    if x > 0
        theta2 = -acos(D);
    else
        theta2 = acos(D);
    end
    theta1 = atan2(y, x) - atan2(l2*sin(theta2), l1 + l2*cos(theta2));

    pos_ang1(i) = theta1;
    pos_ang2(i) = theta2;
end

%Définition des paramètres pour la transition
nb_points_transition = 200;
temps_transition = nb_points_transition * pas_temps;
xTran = 0; yTran = l1 + l2;

% Angles pour la transition
D_centre = (xTran^2 + yTran^2 - l1^2 - l2^2) / (2*l1*l2);
D_centre = min(max(D_centre, -1), 1);
theta2_centre = acos(D_centre);
theta1_centre = atan2(yTran, xTran) - atan2(l2*sin(theta2_centre), l1 + l2*cos(theta2_centre));

% Détection des changements de signe de x
signe_x = sign(xPath);
changements = find(diff(signe_x) ~= 0);

for i = length(changements):-1:1
    indice = changements(i);

    theta_avant = [pos_ang1(indice); pos_ang2(indice)];
    theta_apres = [pos_ang1(indice+1); pos_ang2(indice+1)];

    % Interpolation vers le centre puis du centre vers l'autre coté
    inter1 = zeros(2, nb_points_transition);
    inter2 = zeros(2, nb_points_transition);
    for k = 1:nb_points_transition
        alpha = (k-1)/(nb_points_transition-1);
        inter1(:,k) = (1-alpha)*theta_avant + alpha*[theta1_centre; theta2_centre];
        inter2(:,k) = (1-alpha)*[theta1_centre; theta2_centre] + alpha*theta_apres;
    end

    % Insertion des angles interpolés
    pos_ang1 = [pos_ang1(1:indice), inter1(1,:), inter2(1,:), pos_ang1(indice+1:end)];
    pos_ang2 = [pos_ang2(1:indice), inter1(2,:), inter2(2,:), pos_ang2(indice+1:end)];

    % Insertion du temps pour la transition
    t_transition = (1:2*nb_points_transition) * pas_temps + t(indice);
```

```
    t_post = t(indice+1:end) + 2*temps_transition;
    t = [t(1:indice), t_transition, t_post];
end

% Définition du robot
L1 = Link('d',0,'a',l1,'alpha',0);
L2 = Link('d',0,'a',l2,'alpha',0);
L1.m = m1; L1.r = [lc1 0 0];
L1.I = [0.001 0.001 0.001];
L2.m = m2; L2.r = [lc2 0 0];
L2.I = [0.0008 0.0008 0.0008];
robot = SerialLink([L1 L2], 'name', 'bras2R');
robot.gravity = [0 0 -g];

% Animation du robot
if afficherRobot
    for i = 1:length(t)
        robot.plot([pos_ang1(i), pos_ang2(i)]);
        view(2); axis([-0.4 0.4 0 0.4]); pause(pas_temps);
    end
end

%Dérivées des positions
vit_ang1 = gradient(pos_ang1, pas_temps);
vit_ang2 = gradient(pos_ang2, pas_temps);
acc_ang1 = gradient(vit_ang1, pas_temps);
acc_ang2 = gradient(vit_ang2, pas_temps);

pos_ang = [pos_ang1; pos_ang2];
vit_ang = [vit_ang1; vit_ang2];
acc_ang = [acc_ang1; acc_ang2];
%Calcul du coucle avec la Toolbox
tau = robot.rne(pos_ang', vit_ang', acc_ang');

%Création des consignes pour Simulink
con_pos_ang_1 = timeseries(pos_ang1', t');
con_pos_ang_2 = timeseries(pos_ang2', t');
con_vit_ang_1 = timeseries(vit_ang1', t');
con_vit_ang_2 = timeseries(vit_ang2', t');
con_acc_ang_1 = timeseries(acc_ang1', t');
con_acc_ang_2 = timeseries(acc_ang2', t');
con_couple=timeseries(tau', t');

%conditions Initiales
theta0_1 = pos_ang1(1);
theta0_2 = pos_ang2(1);
thetaDot0_1 = vit_ang1(1);
thetaDot0_2 = vit_ang2(1);
```

```
theta0_1ts = timeseries(theta0_1, 0);
theta0_2ts = timeseries(theta0_2, 0);
thetaDot0_1ts = timeseries(thetaDot0_1, 0);
thetaDot0_2ts = timeseries(thetaDot0_2, 0);

%Création des fichiers csv pour les consignes
donnees_trajectoires = [t', pos_ang1', pos_ang2', vit_ang1', vit_ang2', acc_ang1', acc_ang2'];
writematrix(donnees_trajectoires, 'trajectoire_fixe.csv');
donnees_couples = [t', tau];
writematrix(donnees_couples, 'couple_traj.csv');

%Tracés des différentes figures
figure;
subplot(3,1,1);
plot(t, pos_ang1, 'b', t, pos_ang2, 'r'); grid on;
ylabel('\theta(t) [rad]'); title('Position'); legend('\theta_1','\theta_2');

subplot(3,1,2);
plot(t, vit_ang1, 'b', t, vit_ang2, 'r'); grid on;
ylabel('d\theta(t) [rad/s]'); title('Vitesse');

subplot(3,1,3);
plot(t, acc_ang1, 'b', t, acc_ang2, 'r'); grid on;
ylabel('dd\theta(t) [rad/s²]'); xlabel('Temps [s]');
title('Accélération');

figure;
plot(t, tau(:,1), 'm', 'LineWidth', 1.5); hold on;
plot(t, tau(:,2), 'c', 'LineWidth', 1.5);
xlabel('Temps (s)'); ylabel('Couple (Nm)');
title('Couples moteurs'); legend('\tau_1', '\tau_2'); grid on;


temps_total = length(t) * pas_temps;
fprintf('Temps total de la trajectoire : %.2f secondes (%d points)\n', temps_total, length(t));
% Génération des .h pour STM32
dt = pas_temps;
tableau_consigne="trajectoire_fixe.csv";
tableau_couple_csv = "couple_traj.csv";
tableau_pos1 = "C:\Users\thiba\STM32CubeIDE\workspace_1.18.1\methode1\Core\Inc\consigne_position.h";
tableau_couple= "C:\Users\thiba\STM32CubeIDE\workspace_1.18.1\methode1\Core\Inc\consigne_couple.h";
donnees_consignes=readmatrix(tableau_consigne);
donnees_couples=readmatrix(tableau_couple_csv);

% consigne_position.h
fidc = fopen(tableau_pos1, "w");
fprintf(fidc, "#ifndef CONSIGNE_POSITION_H\n");
fprintf(fidc, "#define CONSIGNE_POSITION_H\n\n");
```

```
fprintf(fidc, "const float consigne_data[%d][6] = {\n", size(donnees_consignes,1));
for i = 1:size(donnees_consignes,1)
    if i < size(donnees_consignes,1)
        fprintf(fidc, "    {%.6ff, %.6ff, %.6ff, %.6ff, %.6ff, %.6ff},\n",donnees_consignes(i,2), donnees_consignes(i,3),
        donnees_consignes(i,4), donnees_consignes(i,5),donnees_consignes(i,6), donnees_consignes(i,7));
    else
        fprintf(fidc, "    {%.6ff, %.6ff, %.6ff, %.6ff, %.6ff, %.6ff}\n",donnees_consignes(i,2), donnees_consignes(i,3)
        ,donnees_consignes(i,4), donnees_consignes(i,5),donnees_consignes(i,6), donnees_consignes(i,7));
    end
end
fprintf(fidc, "};\n");
fprintf(fidc, "const int total_points = %d;\n", size(donnees_consignes,1));
fprintf(fidc, "const float dt = %.6ff;\n", pas_temps);   % ou dt si tu l'as défini
fprintf(fidc, "\n#endif\n");
fclose(fidc);

disp("Fichier " + tableau_pos1 + " généré !");


% consigne_couple.h
fid = fopen(tableau_couple, "w");
fprintf(fid, "#ifndef CONSIGNE_COUPLE_H\n");
fprintf(fid, "#define CONSIGNE_COUPLE_H\n\n");

fprintf(fid, "const float couple_con[%d][2] = {\n", size(donnees_couples,1));
for i = 1:size(donnees_couples,1)
    if i < size(donnees_couples,1)
        fprintf(fid, "    {%.5ff, %.5ff},\n", donnees_couples(i,2), donnees_couples(i,3));
    else
        fprintf(fid, "    {%.5ff, %.5ff}\n",  donnees_couples(i,2), donnees_couples(i,3));
    end
end
fprintf(fid, "};\n");
fprintf(fid, "\n#endif\n");
fclose(fid);

disp("Fichier " + tableau_couple + " généré !");
```

Trajectory drawing function :

```
function [xDraw, yDraw] = dessinTrajectoire()

    % Longueur des bras
    l1 = 0.17;
    l2 = 0.15;
    R = l1 + l2;

    %Figure sur laquelle tracer
    figure;
    title('Trace une trajectoire avec le bouton droit de la souris');
    xlabel('x (m)'); ylabel('y (m)');
    axis equal;
    axis([-R R 0 R]);
    grid on;
    hold on;

    %Délimitation des zones de tracés
    rectangle('Position', [-R -R 2*R 2*R], 'Curvature', [1 1],'EdgeColor', 'r');
    rectangle('Position', [-0.111 -0.111 2*0.111 2*0.111], 'Curvature', [1 1],'EdgeColor', 'r');

    %Coordonnées des points déssinés
    xDraw = [];
    yDraw = [];
    isDrawing = false;

    %Fonction pour déterminer le moment ou on commence à tracer
    set(gcf, 'WindowButtonDownFcn', @startDraw);
    set(gcf, 'WindowButtonMotionFcn', @mouseMove);
    set(gcf, 'WindowButtonUpFcn', @stopDraw);

    uiwait;
    %Début du dessin
    function startDraw(~, ~)
        if strcmp(get(gcf, 'SelectionType'), 'alt')  % clic droit
            isDrawing = true;
            [x, y] = getCurrentPoint();
            ajouterPoint(x, y);
        end
    end
    %Pendant le dessin
    function mouseMove(~, ~)
        if isDrawing
            [x, y] = getCurrentPoint();
            ajouterPoint(x, y);
        end
    end
```

```
    %Fin du dessin
    function stopDraw(~, ~)
        if isDrawing
            isDrawing = false;
            uiresume;
        end
    end

    function [x, y] = getCurrentPoint()
        pt = get(gca, 'CurrentPoint');
        x = pt(1, 1);
        y = pt(1, 2);
    end
    %Lecture des points tracés avec la souris
    function ajouterPoint(x, y)
        r2 = x^2 + y^2;
        R2 = R^2;
        valid = true;
        reason = "";
        %Suppréssion des points tracés dans les zones limites
        if r2 > R2 || y < 0
            valid = false;
            reason = "Point inatteignable pour le bras";
        else
            D = (x^2 + y^2 - l1^2 - l2^2)/(2*l1*l2);
            D = min(max(D, -1), 1);
            theta2 = acos(D);
            if theta2 > (140*pi/180) && theta2 < (230*pi/180)
                valid = false;
                reason = "θ₂ trop proche de 180° le bras va se replier";
            end
        end

        if valid
            xDraw(end+1) = x;
            yDraw(end+1) = y;
            plot(x, y, 'k.');
        else
            plot(x, y, 'r.');
            disp("Point refusé : " + reason + sprintf(" (x=%.3f, y=%.3f)", x, y));
        end
    end
end
```

Determination of the PID corrector

```
clearvars
close all
clc

p=tf('p');
%Paramètres
J1 = 0.0304;
J2=0.0026;
wu = 5;
Mp=60;

%Pour theta1
wi1 = wu / 5;
G1 = 1 / (J1 * p^2);
[gG1,pG1]=bode(G1,wu);
a1 = (1 + sin((-180+Mp-pG1+11.3)*pi/180))/(1-sin((-180+Mp-pG1+11.3)*pi/180));
w11=wu/sqrt(a1);
w21=wu*sqrt(a1);
modG1 = 1 / sqrt( (J1*wu^2)^2 );
K01 = 1/(sqrt(1+(wu/wi1)^2)/(wu/wi1)) * 1/sqrt(a1) * 1/modG1 ;
K1 = K01 * (p/wi1 +1 ) / (p/wi1) * (p/w11 +1 ) / (p/w21 + 1);
L1 = K1 * G1;

%Pour theta2
wi2 = wu / 5;
G2 = 1 / (J2 * p^2);
[gG2,pG2]=bode(G2,wu);
a2 = (1 + sin((-180+Mp-pG2+11.3)*pi/180))/(1-sin((-180+Mp-pG2+11.3)*pi/180));
w12=wu/sqrt(a2);
w22=wu*sqrt(a2);
modG2 = 1 / sqrt( (J2*wu^2)^2);
K02 = 1/(sqrt(1+(wu/wi2)^2)/(wu/wi2)) * 1/sqrt(a2) * 1/modG2 ;
K2 = K02 * (p/wi2 +1 ) / (p/wi2) * (p/w12 +1 ) / (p/w22 + 1);
L2 = K2 * G2;

%Tracé Bode
figure;
margin(L1)
grid on;
title('Boucle ouverte theta 1 K1(s)·G1(s), vérification gain et marge de phase');
figure;
margin(L2)
grid on;
title('Boucle ouverte theta 2 K2(s)·G2(s), vérification gain et marge de phase');

%Boucle fermée
T1 = feedback(K1 * G1, 1);
T2 = feedback(K2 * G2, 1);

%Réponse à un échelon
figure;
step(T1, 10);
grid on;
title('Réponse indicielle en boucle fermée theta 1');
ylabel('\theta(t)');
xlabel('Temps (s)');

figure;
step(T2, 10);
grid on;
title('Réponse indicielle en boucle fermée theta 2');
ylabel('\theta(t)');
xlabel('Temps (s)');

% Bode de la boucle fermée
figure;
bode(T1)
grid on;
title('Bode du système en boucle fermée theta 1');

figure;
bode(T2)
grid on;
title('Bode du système en boucle fermée theta 2');
```

STM32CubeIDE Code

Main code for application of torque + Backup method :

```
#include "main.h"
#include "consigne_position.h"
#include "consigne_couple.h"
#include <stdio.h>
#include <math.h>

TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
TIM_HandleTypeDef htim4;
TIM_HandleTypeDef htim5;
TIM_HandleTypeDef htim10;

UART_HandleTypeDef huart2;
DMA_HandleTypeDef hdma_usart2_rx;
DMA_HandleTypeDef hdma_usart2_tx;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM5_Init(void);
static void MX_TIM3_Init(void);
static void MX_TIM4_Init(void);
static void MX_TIM10_Init(void);

int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}

float r1 = 10.0f;
float r2 = 5.0f;

float step_tour_bras_1 = 2000.0f;
float step_tour_bras_2 = 1000.0f;
float theta_offset1 = 0.0f;
float theta_offset2 = 0.0f;

void homing_moteur1(void) //Permet d'atteindre la position de butée
{
    HAL_GPIO_WritePin(GPIOC, Dir_moteur_1_Pin, GPIO_PIN_SET);
    uint32_t arr = 60000;
    __HAL_TIM_SET_AUTORELOAD(&htim2, arr);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, arr / 2);
    while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_11) == GPIO_PIN_RESET)
    {
        HAL_Delay(1);
    }
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);
}

void homing_moteur2(void) //Permet d'atteindre la position de butée
{
    HAL_GPIO_WritePin(GPIOA, Dir_moteur_2_Pin, GPIO_PIN_SET);
    uint32_t arr = 60000;
    __HAL_TIM_SET_AUTORELOAD(&htim5, arr);
    __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, arr / 2);
    while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_12) == GPIO_PIN_RESET)
    {
        HAL_Delay(1);
    }
    __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, 0);
}
```

```
void pos_init1(void) //Permet d'atteindre la position de référence
{
    HAL_GPIO_WritePin(GPIOC, Dir_moteur_1_Pin, GPIO_PIN_RESET);
    uint32_t arr_homing = 60000;
    __HAL_TIM_SET_AUTORELOAD(&htim2, arr_homing);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, arr_homing / 2);
    HAL_Delay(500);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);
}

void pos_init2(void) //Permet d'atteindre la position de référence
{
    HAL_GPIO_WritePin(GPIOA, Dir_moteur_2_Pin, GPIO_PIN_RESET);
    uint32_t arr_homing = 60000;
    __HAL_TIM_SET_AUTORELOAD(&htim5, arr_homing);
    __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, arr_homing / 2);
    HAL_Delay(350);
    __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, 0);
}


static float go_consigne1(int i) //Permet d'atteindre la première position de la consigne
{
    const float theta_cible  = consigne_data[i][0];
    const float err_ref      = 0.01f;
    uint32_t  arr_con  = 50000;

    while (1)
    {
        int16_t pos = __HAL_TIM_GET_COUNTER(&htim3);
        float theta_mes = (float)pos * 2.0f * M_PI / step_tour_bras_1 / r1;
        float theta1    = theta_mes + theta_offset1;

        float erreur = theta_cible - theta1;
        if (fabsf(erreur) < err_ref) {
            __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);
            return theta1;
        }

        if (erreur > 0.0f)
            HAL_GPIO_WritePin(GPIOC, Dir_moteur_1_Pin, GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOC, Dir_moteur_1_Pin, GPIO_PIN_SET);

        __HAL_TIM_SET_AUTORELOAD(&htim2, arr_con);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, arr_con /2);

        HAL_Delay(5);
    }
}
```

```c
static float go_consigne2(int i) //Permet d'atteindre la première position de la consigne
{
{
    const float theta_cible  = consigne_data[i][1];
    const float err_ref      = 0.01f;
    uint32_t  arr_con  = 50000;

    while (1)
    {
        int16_t pos = __HAL_TIM_GET_COUNTER(&htim4);
        float theta_mes = (float)pos * 2.0f * M_PI / step_tour_bras_2 / r2;
        float theta2    = theta_mes + theta_offset2+1.57;

        float erreur = theta_cible - theta2;
        if (fabsf(erreur) < err_ref) {
            __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, 0);
            return theta2;
        }

        if (erreur > 0.0f)
            HAL_GPIO_WritePin(GPIOA, Dir_moteur_2_Pin, GPIO_PIN_SET);
        else
            HAL_GPIO_WritePin(GPIOA, Dir_moteur_2_Pin, GPIO_PIN_RESET);

        __HAL_TIM_SET_AUTORELOAD(&htim5, arr_con);
        __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, arr_con /2);

        HAL_Delay(5);
        printf("pos_sor2=%.4f rad\r\n", theta2);
    }
}



int main(void)
{
    HAL_Init();

    SystemClock_Config();
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_USART2_UART_Init();
    MX_TIM2_Init();
    MX_TIM5_Init();
    MX_TIM3_Init();
    MX_TIM4_Init();
    MX_TIM10_Init();

    HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
    HAL_TIM_Encoder_Start(&htim4, TIM_CHANNEL_ALL);
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim5, TIM_CHANNEL_1);

    // Réducteurs


    printf("Nouvelle Manip\r\n");
    homing_moteur2();
    homing_moteur1();
    pos_init2();
    pos_init1();

    int16_t pos_s1_ref = __HAL_TIM_GET_COUNTER(&htim3);
    int16_t pos_s2_ref = __HAL_TIM_GET_COUNTER(&htim4);

    float theta1_mesure_ref = (float)pos_s1_ref * 2.0f * M_PI / step_tour_bras_1 / r1;
    float theta2_mesure_ref = (float)pos_s2_ref * 2.0f * M_PI / step_tour_bras_2 / r2;

    theta_offset1 =  - theta1_mesure_ref; //Permet d'avoir la bonne position à chaque nouvelle manipulation
    theta_offset2 =  - theta2_mesure_ref; //Permet d'avoir la bonne position à chaque nouvelle manipulation

    go_consigne1(0);
```

```c
for (int i = 1; i < total_points; i++)
{
    //Méthode de secours
    float theta_atteint1 = go_consigne1(i);
    float theta_atteint2 = go_consigne2(i);
    float erreu1=consigne_data[i][0]-go_consigne1(i);
    float erreur2=consigne_data[i][1]-go_consigne2(i);

    printf("Point %d atteint : consigne=%.4f rad | theta=%.4f rad | erreur=%.4f rad\r\n",
           i, (double)consigne_data[i][0], (double)theta_atteint1,(double)erreur1);

    //Méthode initiale qui n'a finalement pas fonctionnée
    float tau1 = couple_con[i][0]; //Méthode d'application initiale
    float tau2 = couple_con[i][1];

    int16_t pos_s1 = __HAL_TIM_GET_COUNTER(&htim3);
    int16_t pos_s2 = __HAL_TIM_GET_COUNTER(&htim4);

    float theta1_mesure = (float)pos_s1 * 2.0f * M_PI / step_tour_bras_1 / r1;
    float theta2_mesure = (float)pos_s2 * 2.0f * M_PI / step_tour_bras_2 / r2;

    float theta1 = theta1_mesure + theta_offset1; //Permet d'avoir la bonne position à chaque nouvelle manipulation
    float theta2 = theta2_mesure + theta_offset2; //Permet d'avoir la bonne position à chaque nouvelle manipulation

    if (tau1 >= 0)
        HAL_GPIO_WritePin(GPIOC, Dir_moteur_1_Pin, GPIO_PIN_RESET);
    else
        HAL_GPIO_WritePin(GPIOC, Dir_moteur_1_Pin, GPIO_PIN_SET);

    if (tau2 >= 0)
        HAL_GPIO_WritePin(GPIOA, Dir_moteur_2_Pin, GPIO_PIN_RESET);
    else
        HAL_GPIO_WritePin(GPIOA, Dir_moteur_2_Pin, GPIO_PIN_SET);

    float tau_mot1 = fabsf(tau1) / r1;
    float tau_mot2 = fabsf(tau2) / r2;

    float tau_max_mot = 0.4f;
    float vit_max = 6.5f;

    if (tau_mot1 > tau_max_mot)
            tau_mot1 = tau_max_mot;
    if (tau_mot2 > tau_max_mot)
            tau_mot2 = tau_max_mot;
    //Transformation couple en vitesse
    float vit1 = (tau_mot1 / tau_max_mot) * vit_max;
    float vit2 = (tau_mot2 / tau_max_mot) * vit_max;
    //Transformation vitesse en fréquence
    float freq1 = vit1 * step_tour_bras_1 / (2.0f * M_PI);
    float freq2 = vit2 * step_tour_bras_2 / (2.0f * M_PI);
    //Transformation fréquence en ARR
    uint32_t arr1, arr2;
    if (freq1 > 1e-2f)
        arr1 = (uint32_t)(84000000.0f / (freq1 * 2.0f));
    else
        arr1 = 0;

    if (freq2 > 1e-2f)
        arr2 = (uint32_t)(84000000.0f / (freq2 * 2.0f));
    else
        arr2 = 0;
    //Application de l'ARR à chaque moteur
    __HAL_TIM_SET_AUTORELOAD(&htim2, arr1);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, arr1/2);

    __HAL_TIM_SET_AUTORELOAD(&htim5, arr2);
    __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, arr2/2);


    float erreur1 = consigne_data[i][0] - theta1;

    printf("t=%.3f s | pos_sor1=%.4f rad | pos_con1=%.4f rad\r\n",i * dt, theta2, consigne_data[i][1]);

    HAL_Delay(dt * 1000);
```

Main code for Application of the dynamic equation :

```
float PID1(float pos_1_mes, int k)
{
    float erreur = consigne_data[k][0] - pos_1_mes;

    inte1 += erreur * dt;
    float deriv = (erreur - erreur_prec1)/ dt;

    float sortie_pid = Kp1 * erreur  + Ki1 * inte1 + Kd1 * deriv;

    erreur_prec1 = erreur;

    return sortie_pid;
}

float PID2(float pos_2_mes, int k)
{
    float erreur = consigne_data[k][1] - pos_2_mes;

    inte2 += erreur * dt;
    float deriv = (erreur - erreur_prec2)/ dt;

    float sortie_pid = Kp2 * erreur  + Ki2 * inte2 + Kd2 * deriv;

    erreur_prec2 = erreur;

    return sortie_pid;
}

void dynamique_inverse(float theta1, float theta2,float dtheta1, float dtheta2,float ddtheta1, float ddtheta2,float *tau1, float *tau2)
{

    float M11 = I1 + I2 + m1*lc1*lc1 + m2*(l1*l1 + lc2*lc2 + 2*l1*lc2*cosf(theta2));
    float M12 = I2 + m2*(lc2*lc2 + l1*lc2*cosf(theta2));
    float M21 = M12;
    float M22 = I2 + m2*lc2*lc2;

    float C1 = -m2*l1*lc2*sinf(theta2)*(2*dtheta1*dtheta2 + dtheta2*dtheta2);
    float C2 =  m2*l1*lc2*sinf(theta2)*dtheta1*dtheta1;

    float G1 = (m1*lc1 + m2*l1)*g*cosf(theta1) + m2*lc2*g*cosf(theta1 + theta2);
    float G2 = m2*lc2*g*cosf(theta1 + theta2);

    *tau1 = M11*ddtheta1 + M12*ddtheta2 + C1 + G1;
    *tau2 = M21*ddtheta1 + M22*ddtheta2 + C2 + G2;
```

```
    for (int i = 1; i < total_points; i++)
    {
        int16_t pos_s1 = __HAL_TIM_GET_COUNTER(&htim3);
        int16_t pos_s2 = __HAL_TIM_GET_COUNTER(&htim4);

        float theta1_mesure = (float)pos_s1 * 2.0f * M_PI / step_tour_bras_1 / r1;
        float theta2_mesure = (float)pos_s2 * 2.0f * M_PI / step_tour_bras_2 / r2;

        float theta1 = (theta1_mesure + theta_offset1);
        float theta2 = theta2_mesure + theta_offset2+1.57;

        float dtheta1 = consigne_data[i][2];
        float dtheta2 = consigne_data[i][3];
        float ddtheta1 = consigne_data[i][4];
        float ddtheta2 = consigne_data[i][5];

        float tau1, tau2;

        dynamique_inverse(theta1, theta2, dtheta1, dtheta2, ddtheta1, ddtheta2, &tau1, &tau2);

        tau1 +=PID1(theta1,i);
        tau2 +=PID2(theta2,i);
```